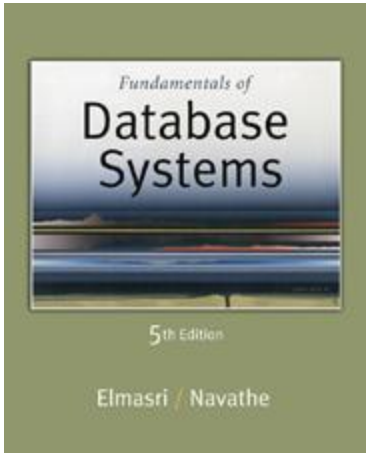
Functional Dependencies and Normalization for Relational Databases



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Chapter Outline

- 2 Functional Dependencies (FDs)
  - 2.1 Definition of FD
  - 2.2 Inference Rules for FDs
  - 2.3 Equivalence of Sets of FDs
  - 2.4 Minimal Sets of FDs
- 3 Normal Forms Based on Primary Keys
  - 3.1 Normalization of Relations
  - 3.2 Practical Use of Normal Forms
  - 3.3 Definitions of Keys and Attributes Participating in Keys
  - 3.4 First Normal Form
  - 3.5 Second Normal Form
  - 3.6 Third Normal Form

- 4 General Normal Form Definitions (For Multiple Keys)
- 5 BCNF (Boyce-Codd Normal Form)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Functional Dependencies

- A functional dependency is a constraint between two sets of attributes from the database.
- X -> Y holds if whenever two tuples have the same value for X, they *must have* the same value for Y
  - For any two tuples t1 and t2 in any relation instance r(R): If t1[X]=t2[X], *then* t1[Y]=t2[Y] is true
- X -> Y in R specifies a *constraint* on all relation instances r(R)
- Written as X -> Y; can be displayed graphically on a relation schema. (denoted by the arrow:).
- FDs are derived from the real-world constraints on the attributes

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Examples of FD constraints (1)

- Social security number determines employee name
  - SSN -> ENAME
- Project number determines project name and location
  - PNUMBER -> {PNAME, PLOCATION}
- Employee ssn and project number determines the hours per week that the employee works on the project
  - {SSN, PNUMBER} -> HOURS

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

TEACH

Teacher	Course	Text
Smith	Data Structures	Bartram
Smith	Data Management	Martin
Hall	Compilers	Hoffman
Brown	Data Structures	Horowitz

Figure 15.7
A relation state of TEACH with a *possible* functional dependency TEXT → COURSE. However, TEACHER → COURSE is ruled out.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Examples of FD constraints (2)

- An FD is a property of the attributes in the schema R
- The constraint must hold on *every* relation instance r(R)
- If K is a key of R, then K functionally determines all attributes in R
  - (since we never have two distinct tuples with t1[K]=t2[K])

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Find out all the possible FDs in the following example

A	B	C	D
a1	b1	c1	d1
a1	b2	c2	d2
a2	b2	c2	d3
a3	b3	c4	d3

- Here, the following FDs may hold because the four tuples in the current extension have no violation of these constraints: B → C; C →B;{A, B} → C; {A, B} → D; and {C, D} → B. However, the following do not hold because we already have violations of them in the given extension: A → B (tuples 1 and 2 violate this constraint); B → A (tuples 2 and 3 violate this constraint); D → C (tuples 3 and 4 violate it).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.2 Inference Rules for FDs (1)

- Given a set of FDs F, we can **infer** additional FDs that hold whenever the FDs in F hold
- Armstrong's inference rules:
  - IR1. (**Reflexive**) If Y *subset-of* X, then X -> Y
  - IR2. (**Augmentation**) If X -> Y, then XZ -> YZ
    - (Notation: XZ stands for X U Z)
  - IR3. (**Transitive**) If X -> Y and Y -> Z, then X -> Z

- IR1, IR2, IR3 form a **sound** and **complete** set of inference rules
  - These are rules hold and all other rules that hold can be deduced from these

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (2)

- Some additional inference rules that are useful:
  - **Decomposition:** If X -> YZ, then X -> Y and X -> Z
  - **Union:** If X -> Y and X -> Z, then X -> YZ
  - **Psuedotransitivity:** If X -> Y and WY -> Z, then WX -> Z

- The last three inference rules, as well as any other inference rules, can be deduced from IR1, IR2, and IR3 (completeness property)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Proofs for inference Rules IR1 to IR3

Proof of IR1. Suppose that $X \supseteq Y$ and that two tuples t_1 and t_2 exist in some relation instance r of R such that $t_1[X] = t_2[X]$. Then $t_1[Y] = t_2[Y]$ because $X \supseteq Y$; hence, $X \rightarrow Y$ must hold in r.

Proof of IR2 (by contradiction). Assume that $X \rightarrow Y$ holds in a relation instance r of R but that $XZ \rightarrow YZ$ does not hold. Then there must exist two tuples t_1 and t_2 in r such that (1) $t_1[X] = t_2[X]$, (2) $t_1[Y] = t_2[Y]$, (3) $t_1[XZ] = t_2[XZ]$, and (4) $t_1[YZ] \neq t_2[YZ]$. This is not possible because from (1) and (3) we deduce (5) $t_1[Z] = t_2[Z]$, and from (2) and (5) we deduce (6) $t_1[YZ] = t_2[YZ]$, contradicting (4).

Proof of IR3. Assume that (1) $X \rightarrow Y$ and (2) $Y \rightarrow Z$ both hold in a relation r. Then for any two tuples t_1 and t_2 in r such that $t_1[X] = t_2[X]$, we must have (3) $t_1[Y] = t_2[Y]$, from assumption (1); hence we must also have (4) $t_1[Z] = t_2[Z]$ from (3) and assumption (2); thus $X \rightarrow Z$ must hold in r.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Proof of IR4 (Using IR1 through IR3).

1. $X \rightarrow YZ$ (given).
2. $YZ \rightarrow Y$ (using IR1 and knowing that $YZ \supseteq Y$).
3. $X \rightarrow Y$ (using IR3 on 1 and 2).

Proof of IR5 (using IR1 through IR3).

1. $X \rightarrow Y$ (given).
2. $X \rightarrow Z$ (given).
3. $X \rightarrow XY$ (using IR2 on 1 by augmenting with X; notice that $XX = X$).
4. $XY \rightarrow YZ$ (using IR2 on 2 by augmenting with Y).
5. $X \rightarrow YZ$ (using IR3 on 3 and 4).

Proof of IR6 (using IR1 through IR3).

1. $X \rightarrow Y$ (given).
2. $WY \rightarrow Z$ (given).
3. $WX \rightarrow WY$ (using IR2 on 1 by augmenting with W).
4. $WX \rightarrow Z$ (using IR3 on 3 and 2).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Inference Rules for FDs (3)

- **Closure** of a set F of FDs is the set F^+ of all FDs that can be inferred from F
- **Closure** of a set of attributes X with respect to F is the set X^+ of all attributes that are functionally determined by X
- X^+ can be calculated by repeatedly applying IR1, IR2, IR3 using the FDs in F

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Definition. For each such set of attributes X, we determine the set X^+ of attributes that are functionally determined by X based on F; X^+ is called the **closure of X under F**. Algorithm 16.1 can be used to calculate X^+.

Algorithm 16.1. Determining X^+, the Closure of X under F

Input: A set F of FDs on a relation schema R, and a set of attributes X, which is a subset of R.

$X^+ := X$;
repeat
    old$X^+ := X^+$;
    for each functional dependency $Y \rightarrow Z$ in F do
        if $X^+ \supseteq Y$ then $X^+ := X^+ \cup Z$;
until ($X^+ =$ oldX^+);

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- If $F = \{\text{Ssn} \rightarrow \text{Ename},$
  $\text{Pnumber} \rightarrow \{\text{Pname, Plocation}\},$
  $\{\text{Ssn, Pnumber}\} \rightarrow \text{Hours}\}$

- Then

Using Algorithm 16.1, we calculate the following closure sets with respect to F:

$\{\text{Ssn}\}^+ = \{\text{Ssn, Ename}\}$
$\{\text{Pnumber}\}^+ = \{\text{Pnumber, Pname, Plocation}\}$
$\{\text{Ssn, Pnumber}\}^+ = \{\text{Ssn, Pnumber, Ename, Pname, Plocation, Hours}\}$

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.3 Equivalence of Sets of FDs

- Two sets of FDs F and G are **equivalent** if:
  - Every FD in F can be inferred from G, and
  - Every FD in G can be inferred from F
  - Hence, F and G are equivalent if $F^+ = G^+$
- Definition (**Covers**):
  - F **covers** G if every FD in G can be inferred from F
    - (i.e., if G^+ *subset-of* F^+)
- F and G are equivalent if F covers G and G covers F
- There is an algorithm for checking equivalence of sets of FDs

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

1. If all FDs of FD1 can be derived from FDs present in FD2, we can say that FD2 subset of FD1.
2. If all FDs of FD2 can be derived from FDs present in FD1, we can say that FD1 subset of FD2.
3. If 1 and 2 both are true, FD1=FD2

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Check whether the following FDs are equivalent or not:

- A relation R(A,B,C,D) having two FD sets FD1 = {A->B, B->C, AB->D} and FD2 = {A->B, B->C, A->C, A->D}
- A relation R2(A,B,C,D) having two FD sets FD1 = {A->B, B->C,A->C} and FD2 = {A->B, B->C, A->D}

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2.4 Minimal Sets of FDs (1)

- A set of FDs is **minimal** if it satisfies the following conditions:
  1. Every dependency in F has a single attribute for its RHS.
  2. We cannot remove any dependency from F and have a set of dependencies that is equivalent to F.
  3. We cannot replace any dependency X -> A in F with a dependency Y -> A, where Y proper-subset-of X (Y subset-of X) and still have a set of dependencies that is equivalent to F.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Minimal Sets of FDs (2)

- Every set of FDs has an equivalent minimal set
- There can be several equivalent minimal sets
- There is no simple algorithm for computing a minimal set of FDs that is equivalent to a set F of FDs
- To synthesize a set of relations, we assume that we start with a set of dependencies that is a minimal set

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Algorithm 16.2. Finding a Minimal Cover F for a Set of Functional Dependencies E

Input: A set of functional dependencies E.

1. Set $F := E$.
2. Replace each functional dependency $X \rightarrow \{A_1, A_2, \ldots, A_n\}$ in F by the n functional dependencies $X \rightarrow A_1, X \rightarrow A_2, \ldots, X \rightarrow A_n$.
3. For each functional dependency $X \rightarrow A$ in F
   for each attribute B that is an element of X
   if $\{ \{F - \{X \rightarrow A\}\} \cup \{ (X - \{B\}) \rightarrow A\} \}$ is equivalent to F
   then replace $X \rightarrow A$ with $(X - \{B\}) \rightarrow A$ in F.
4. For each remaining functional dependency $X \rightarrow A$ in F
   if $\{F - \{X \rightarrow A\}\}$ is equivalent to F,
   then remove $X \rightarrow A$ from F.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Step 1: Split the Functional Dependencies to have single attribute on the right-hand side
- Step 2: Remove Redundant FDs
- Step 3: Remove Extraneous Attributes on the left-hand side if you can derive the lefthand side attributes from the closure
- eg: if we have an FD as AB-> E then in the last step check whether you are getting B in A+ then remove B to make the FD as A->E or if you are getting A in B+ then you can remove A to make the FD as B->E, (do not consider AB->E while deriving A+ or B+)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Exercise Problems

Find the minimal cover

1. E :{B → A, D → A, AB → D}
2. F : {A->B, B ->C, D->ABC, AC-> D}

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- E :{B → A, D → A, AB → D}

1. No decomposition possible
2. No redundant FDs
3. B+={A,B} without considering AB->D, since A is in B+, A can be removed from AB->D
4. So we get {B → A, D → A, B → D}.
5. using the transitive rule on B →D and D→A, we derive B →A. Hence B →A is redundant in E and can be eliminated.
6. Therefore, the minimal cover of E is {B → D,D → A}

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- F : {A->B, B ->C, D->ABC, AC-> D}

1. Applying decomposition on D->ABC, we get D->A, D->B, D->C
2. Using transitive property on D->A ,A->B, D->B we can remove D->B, hence we get

   F' : {A->B, B ->C, D->A, D->C, AC-> D}

3. A+={A,B,C} and C+ = { C }, since A+ includes C, we can remove C from AC->D to make A->D
4. F' : {A->B, B ->C, D->A, D->C, A-> D}
5. You can remove D->A and A->D
6. Minimal cover is {A->B, B ->C, D->A, D->C, A-> D}

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Algorithm to determine the key of a relation

- In Algorithm 16.2(a), we start by setting K to all the attributes of R; we then remove one attribute at a time and check whether the remaining attributes still form a superkey.
- Notice, that Algorithm 16.2(a) determines only one key out of the possible candidate keys for R; the key returned depends on the order in which attributes are removed from R in step 2

Algorithm 16.2(a). Finding a Key K for R Given a set F of Functional Dependencies

Input: A relation R and a set of functional dependencies F on the attributes of R.

1. Set $K := R$.
2. For each attribute A in K

   {compute $(K - A)^+$ with respect to F;

   if $(K - A)^+$ contains all the attributes in R, then set $K := K - \{A\}$ };

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3 Normal Forms Based on Primary Keys

- Normalization of Relations
- Practical Use of Normal Forms
- Definitions of Keys and Attributes Participating in Keys
- First Normal Form
- Second Normal Form
- Third Normal Form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.1 Normalization of Relations (1)

- **Normalization:**
  - The process of decomposing unsatisfactory "bad" relations by breaking up their attributes into smaller relations
- **Normalization of data**
  - can be considered a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of (1) minimizing redundancy and (2) minimizing the insertion, deletion, and update anomalies
- **Normal form:**
  - Condition using keys and FDs of a relation to certify whether a relation schema is in a particular normal form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Thus, the normalization procedure provides database designers with the following:
  - A formal framework for analyzing relation schemas based on their keys and on the functional dependencies among their attributes
  - A series of normal form tests that can be carried out on individual relation schemas so that the relational database can be normalized to any desired degree
- **Definition**. The normal form of a relation refers to the highest normal form condition that it meets, and hence indicates the degree to which it has been normalized.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalization of Relations (2)

- 2NF, 3NF, BCNF
  - based on keys and FDs of a relation schema
- 4NF
  - based on keys, multi-valued dependencies : MVDs; 5NF based on keys, join dependencies : JDs (Chapter 11)
- Additional properties may be needed to ensure a good relational design (lossless join, dependency preservation; Chapter 11)

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.2 Practical Use of Normal Forms

- **Normalization** is carried out in practice so that the resulting designs are of high quality and meet the desirable properties
- The practical utility of these normal forms becomes questionable when the constraints on which they are based are *hard to understand* or to *detect*
- The database designers *need not* normalize to the highest possible normal form
  - (usually up to 3NF, BCNF or 4NF)
- **Denormalization**:
  - The process of storing the join of higher normal form relations as a base relation—which is in a lower normal form

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.3 Definitions of Keys and Attributes Participating in Keys (1)

- A **superkey** of a relation schema R = {A1, A2, ..., An} is a set of attributes S *subset-of* R with the property that no two tuples t1 and t2 in any legal relation state r of R will have t1[S] = t2[S]

- A **key** K is a **superkey** with the *additional property* that removal of any attribute from K will cause K not to be a superkey any more.
- Eg:{Ssn} is a key for EMPLOYEE, whereas {Ssn}, {Ssn, Ename}, {Ssn, Ename, Bdate}, and any set of attributes that includes Ssn are all superkeys.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Definitions of Keys and Attributes Participating in Keys (2)

- If a relation schema has more than one key, each is called a **candidate** key.
  - One of the candidate keys is *arbitrarily* designated to be the **primary key**, and the others are called **secondary keys**.
  - {Ssn} is the only candidate key for EMPLOYEE,so it is also the primary key.
- A **Prime attribute** must be a member of *some* candidate key
- A **Nonprime attribute** is not a prime attribute—that is, it is not a member of any candidate key.
- both Ssn and Pnumber are prime attributes of WORKS_ON, whereas other attributes of WORKS_ON are nonprime.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3.2 First Normal Form

- Disallows
  - composite attributes
  - multivalued attributes
  - **nested relations**; attributes whose values for an *individual tuple* are non-atomic

- Considered to be part of the definition of relation
- The only attribute values permitted by 1NF are **single atomic (or indivisible) values**.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalization into 1NF

- Solution 1

(a)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations

(b)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

(c)

DEPARTMENT

Dname	Dnumber	Dmgr_ssn	Dlocation
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

Figure 10.8
Normalization into 1NF. (a) A relation schema that is not in 1NF. (b) Example state of relation DEPARTMENT. (c) 1NF version of the same relation with redundancy.

Solution 2

- If a maximum number of values is known for the attribute—for example, if it is known that at most three locations can exist for a department—replace the DIocations attribute by three atomic attributes: DIocation1, DIocation2, and DIocation3.
- This solution has the disadvantage of introducing NULL values if most departments have fewer than three locations.
- It further introduces spurious semantics about the ordering among the location values that is not originally intended.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Solution 3 which is preferred

- Remove the attribute Dlocations that violates 1NF and place it in a separate relation DEPT_LOCATIONS along with the primary key Dnumber of DEPARTMENT. The primary key of this relation is the combination {Dnumber, Dlocation}.
- A distinct tuple in DEPT_LOCATIONS exists for each location of a department.
- This decomposes the non-1NF relation into two 1NF relations

DEPARTMENT

Dname	Dnumber	Dmgr_ssn
Research	5	333445555
Administration	4	987654321
Headquarters	1	888665555

DEPT_LOCATIONS

Dnumber	Dlocation
1	Houston
4	Stafford
5	Bellaire
5	Sugarland
5	Houston

- The EMP_PROJ relation could appear if nesting is allowed, then the schema of this EMP_PROJ relation can be represented as follows:

EMP_PROJ(Ssn, Ename,{PROJS(Pnumber, Hours)})

where the attribute PROJS is multivalued.

- Decomposition and primary key propagation yield the schemas EMP_PROJ1 and EMP_PROJ2

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalization nested relations into 1NF

(a)

EMP_PROJ

		Projs	
Ssn	Ename	Pnumber	Hours

(b)

EMP_PROJ

Ssn	Ename	Pnumber	Hours
123456789	Smith, John B.	1	32.5
		2	7.5
666884444	Narayan, Ramesh K.	3	40.0
453453453	English, Joyce A.	1	20.0
		2	20.0
333445555	Wong, Franklin T.	2	10.0
		3	10.0
		10	10.0
		20	10.0
999887777	Zelaya, AliciaJ.	30	30.0
		10	10.0
987987987	Jabbar, Ahmad V.	10	35.0
		30	5.0
987654321	Wallace, Jennifer S.	30	20.0
		20	15.0
888665555	Borg, James E.	20	NULL

(c)

EMP_PROJ1

Ssn	Ename

EMP_PROJ2

Ssn	Pnumber	Hours

Figure 10.9
Normalizing nested relations into 1NF. (a) Schema of the EMP_PROJ relation with a *nested relation* attribute PROJS. (b) Example extension of the EMP_PROJ relation showing nested relations within each tuple. (c) Decomposition of EMP_PROJ into relations EMP_PROJ1 and EMP_PROJ2 by propagating the primary key.

3.3 Second Normal Form (1)

- Uses the concepts of **FDs, primary key**
- Definitions
  - **Prime attribute:** An attribute that is member of the primary key K
  - **Full functional dependency:** a FD Y -> Z where removal of any attribute from Y means the FD does not hold any more
- Examples:
  - {SSN, PNUMBER} -> HOURS is a full FD since neither SSN -> HOURS nor PNUMBER -> HOURS hold
  - {SSN, PNUMBER} -> ENAME is not a full FD (it is called a partial dependency) since SSN -> ENAME also holds

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Second Normal Form (2)

- **Definition**: A relation schema R is in **second normal form (2NF)** if every non-prime attribute A in R is fully functionally dependent on the primary key
- R can be decomposed into 2NF relations via the process of 2NF normalization
- Find out the FD is in the following relation

EMP_PROJ

Ssn	Pnumber	Hours	Ename	Pname	Plocation

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalizing into 2NF and 3NF

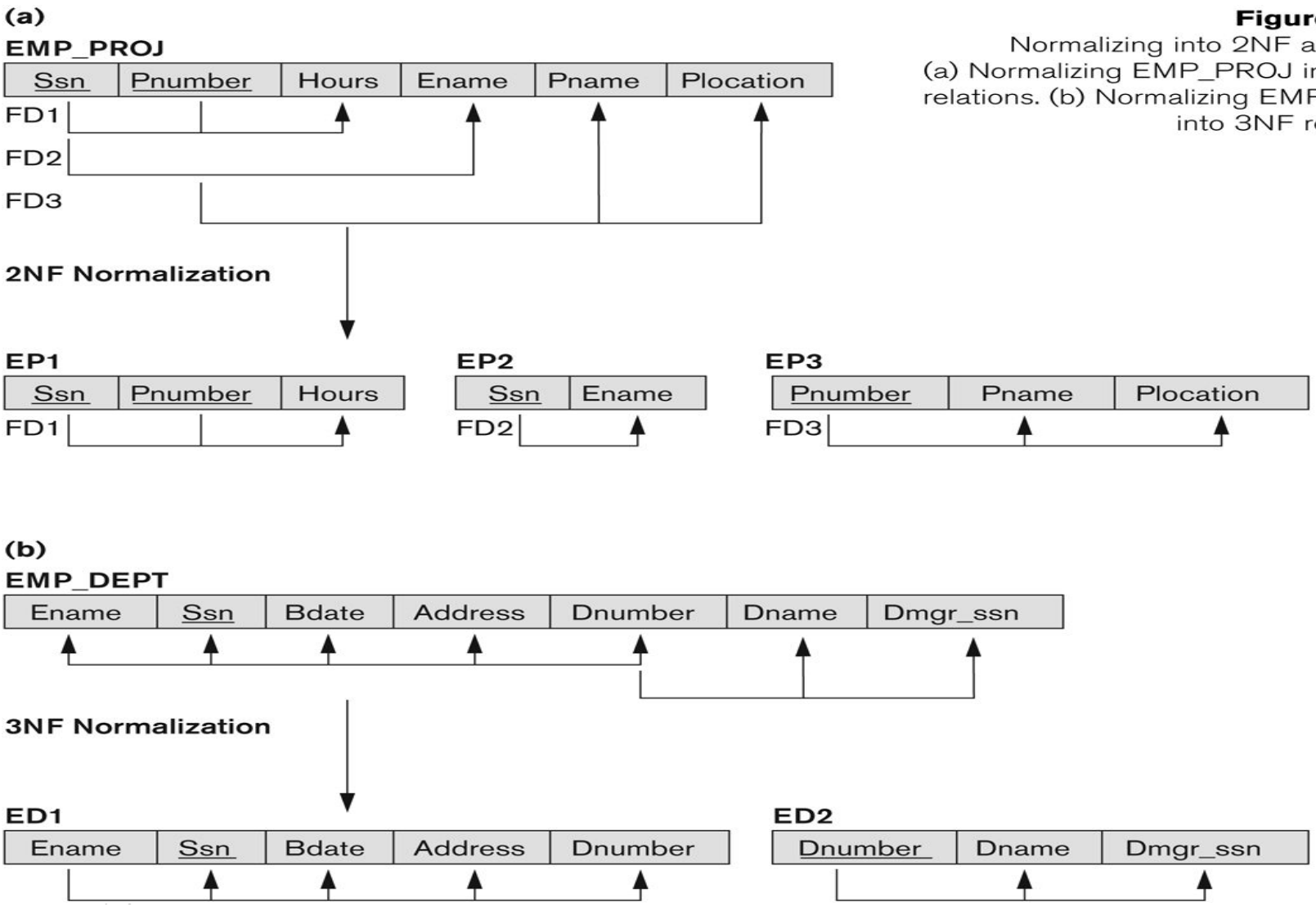


Figure 10.10
Normalizing into 2NF and 3NF. (a) Normalizing EMP_PROJ into 2NF relations. (b) Normalizing EMP_DEPT into 3NF relations.

3.4 Third Normal Form (1)

- Definition:
  - **Transitive functional dependency:** a FD X -> Z that can be derived from two FDs X -> Y and Y -> Z
  - Y should not be non prime attribute when deriving transitive attribute
- Examples:
  - SSN -> DMGRSSN is a **transitive** FD
    - Since SSN -> DNUMBER and DNUMBER -> DMGRSSN hold
    - DNUMBER is non prime attribute in EMPLOYEE table
  - SSN -> ENAME is **non-transitive**
    - Since there is no set of attributes X where SSN -> X and X -> ENAME

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Third Normal Form (2)

- A relation schema R is in **third normal form (3NF)** if it is in 2NF *and* no non-prime attribute A in R is transitively dependent on the primary key
- R can be decomposed into 3NF relations via the process of 3NF normalization
- NOTE:
  - In X -> Y and Y -> Z, with X as the primary key, we consider this a problem only if Y is not a candidate key.
  - When Y is a candidate key, there is no problem with the transitive dependency .
  - E.g., Consider EMP (SSN, Emp#, Salary).
    - Here, SSN -> Emp# -> Salary and Emp# is a candidate key.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Consider the relation schema LOTS, which describes parcels of land for sale in various counties of a state.
- Suppose that there are two candidate keys: Property_id# and {County_name, Lot#}; that is, lot numbers are unique only within each county, but Property_id# numbers are unique across counties for the entire state.
- Find out the FDs in the following relation LOTS

Property_id#	County_name	Lot#	Area	Price	Tax_rate

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalization into 2NF and 3NF

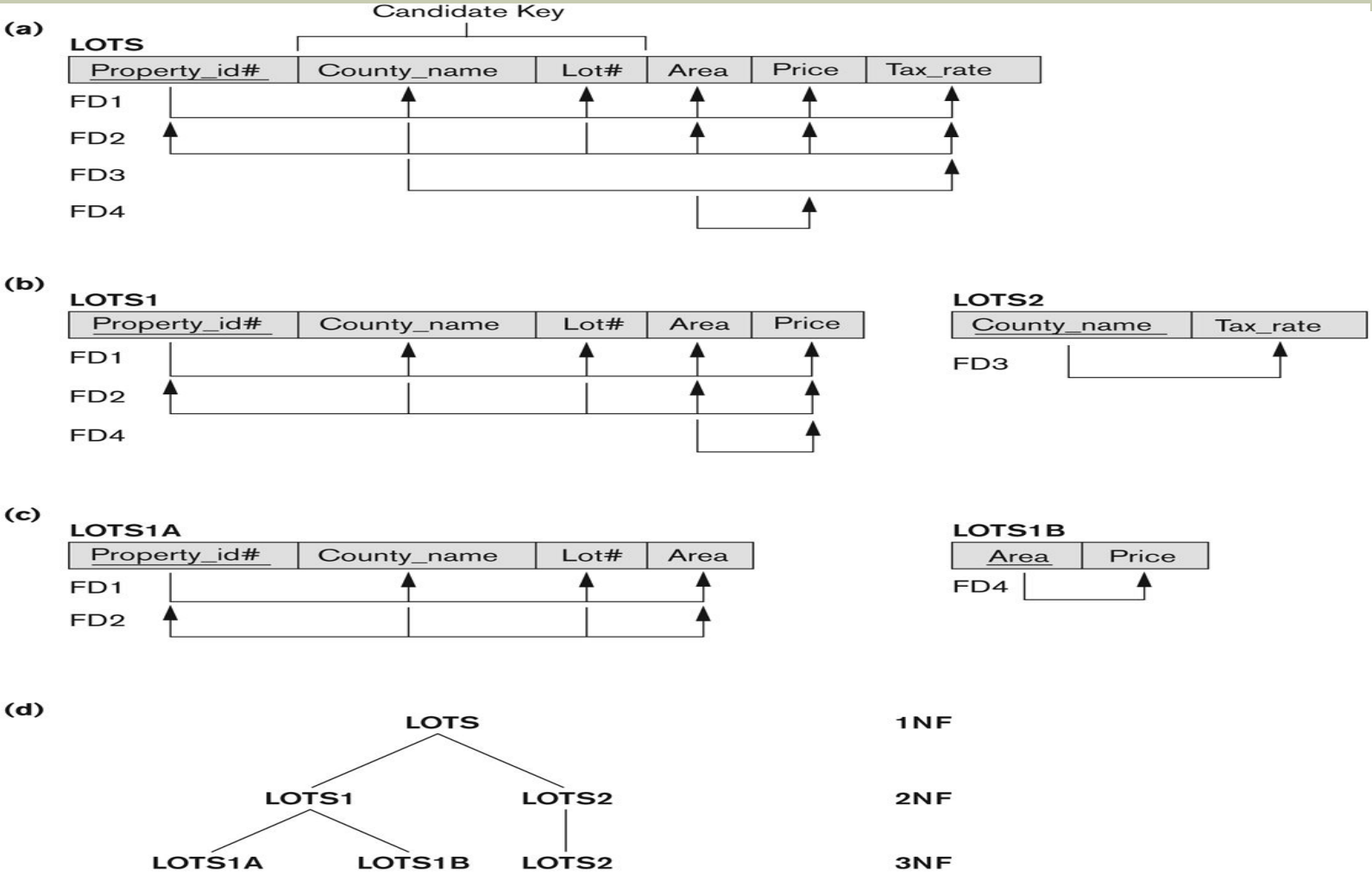


Figure 10.11
Normalization into 2NF and 3NF. (a) The LOTS relation with its functional dependencies FD1 through FD4. (b) Decomposing into the 2NF relations LOTS1 and LOTS2. (c) Decomposing LOTS1 into the 3NF relations LOTS1A and LOTS1B. (d) Summary of the progressive normalization of LOTS.

Table 15.1 Summary of Normal Forms Based on Primary Keys and Corresponding Normalization

Normal Form	Test	Remedy (Normalization)
First (1NF)	Relation should have no multivalued attributes or nested relations.	Form new relations for each multivalued attribute or nested relation.
Second (2NF)	For relations where primary key contains multiple attributes, no nonkey attribute should be functionally dependent on a part of the primary key.	Decompose and set up a new relation for each partial key with its dependent attribute(s). Make sure to keep a relation with the original primary key and any attributes that are fully functionally dependent on it.
Third (3NF)	Relation should not have a nonkey attribute functionally determined by another nonkey attribute (or by a set of nonkey attributes). That is, there should be no transitive dependency of a nonkey attribute on the primary key.	Decompose and set up a relation that includes the nonkey attribute(s) that functionally determine(s) other nonkey attribute(s).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normal Forms Defined Informally

- 1st normal form
  - All attributes depend on **the key**
- 2nd normal form
  - All attributes depend on **the whole key**
- 3rd normal form
  - All attributes depend on **nothing but the key**

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

4 General Normal Form Definitions (For Multiple Keys) (1)

- Design our relation schemas so that they have neither partial nor transitive dependencies because these types of dependencies cause the update anomalies
- The above definitions consider the primary key only
- As a general definition of prime attribute, an attribute that is part of any candidate key will be considered as prime.
- Partial and full functional dependencies and transitive dependencies will now be considered with respect to all candidate keys of a relation.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

General Definition of Second Normal Form

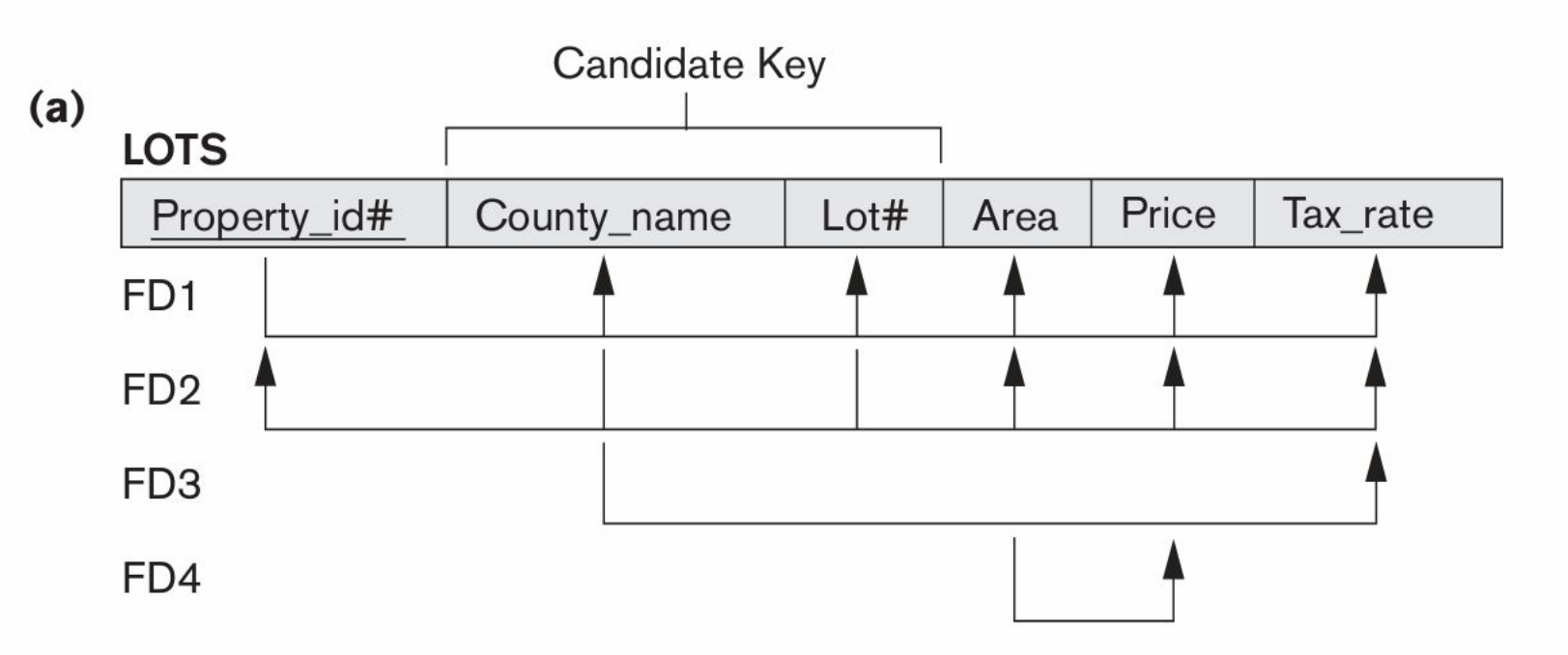
- **Definition**: A relation schema R is in second normal form (2NF) if every non prime attribute A in R is not partially dependent on any key of R.
- The test for 2NF involves testing for functional dependencies whose left-hand side attributes are part of the primary key.
- Suppose if we introduce the below dependencies to LOTS relations

FD3: County_name → Tax_rate

FD4: Area → Price

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

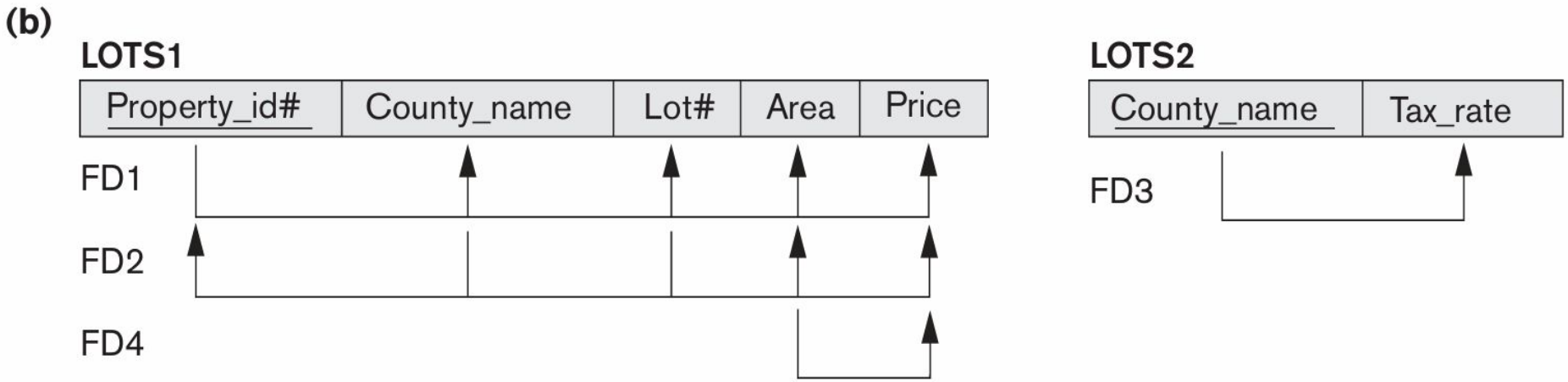
- The LOTS relation schema violates the general definition of 2NF because Tax_rate is partially dependent on the candidate key {County_name, Lot#}, due to FD3.



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Normalizing into 2NF

- To normalize LOTS into 2NF, we decompose it into the two relations LOTS1 and LOTS2
- Notice that FD4 does not violate 2NF and is carried over to LOTS1



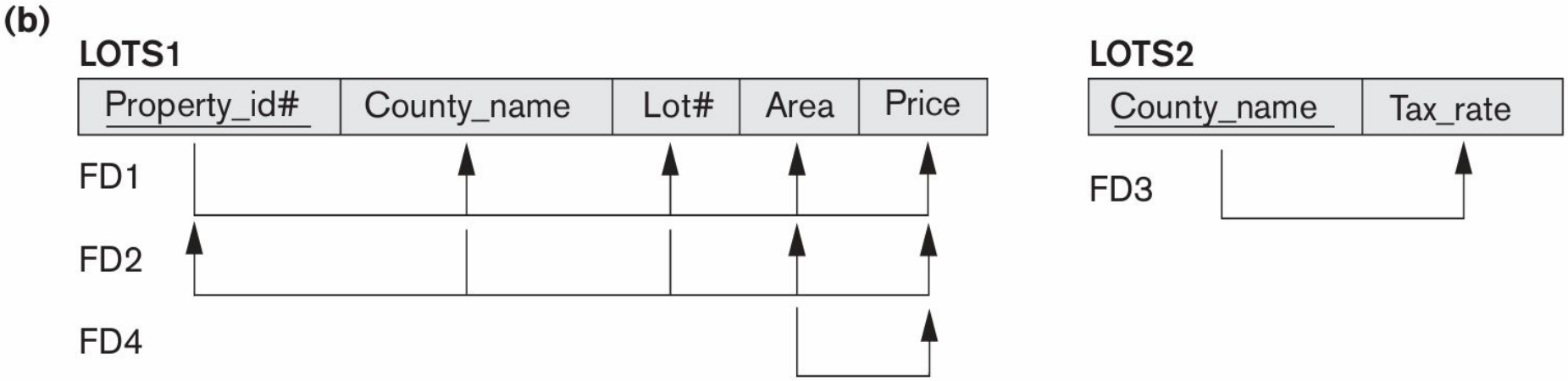
Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

General Normal Form Definitions (2)

- Definition:
  - **Superkey** of relation schema R - a set of attributes S of R that contains a key of R
  - A relation schema R is in **third normal form (3NF)** if whenever a nontrivial FD X -> A holds in R, then either:
    - (a) X is a superkey of R, or
    - (b) A is a prime attribute of R
- NOTE: Boyce-Codd normal form disallows condition (b) above

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

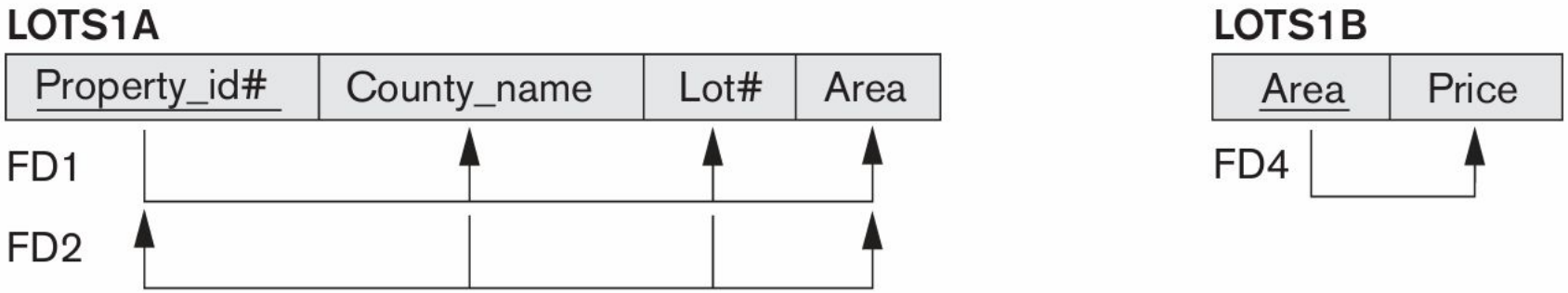
- According to this definition, LOTS2 is in 3NF



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- However, FD4 in LOTS1 violates 3NF because Area is not a superkey and Price is not a prime attribute in LOTS1.
- To normalize LOTS1 into 3NF, we decompose it into the relation schemas LOTS1A and LOTS1B

(c)



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

5 BCNF (Boyce-Codd Normal Form)

- A relation schema R is in **Boyce-Codd Normal Form (BCNF)** if whenever an **FD X -> A** holds in R, then **X is a superkey** ofR
- Each normal form is strictly stronger than the previous one
  - Every 2NF relation is in 1NF
  - Every 3NF relation is in 2NF
  - Every BCNF relation is in 3NF
- There exist relations that are in 3NF but not in BCNF
- The goal is to have each relation in BCNF (or 3NF)
- The formal definition of BCNF differs from the definition of3NF in that condition (b) of3NF,which allows Ato be prime, is absent from BCNF.
- That makes BCNF a stronger normal form compared to 3NF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Boyce-Codd normal form

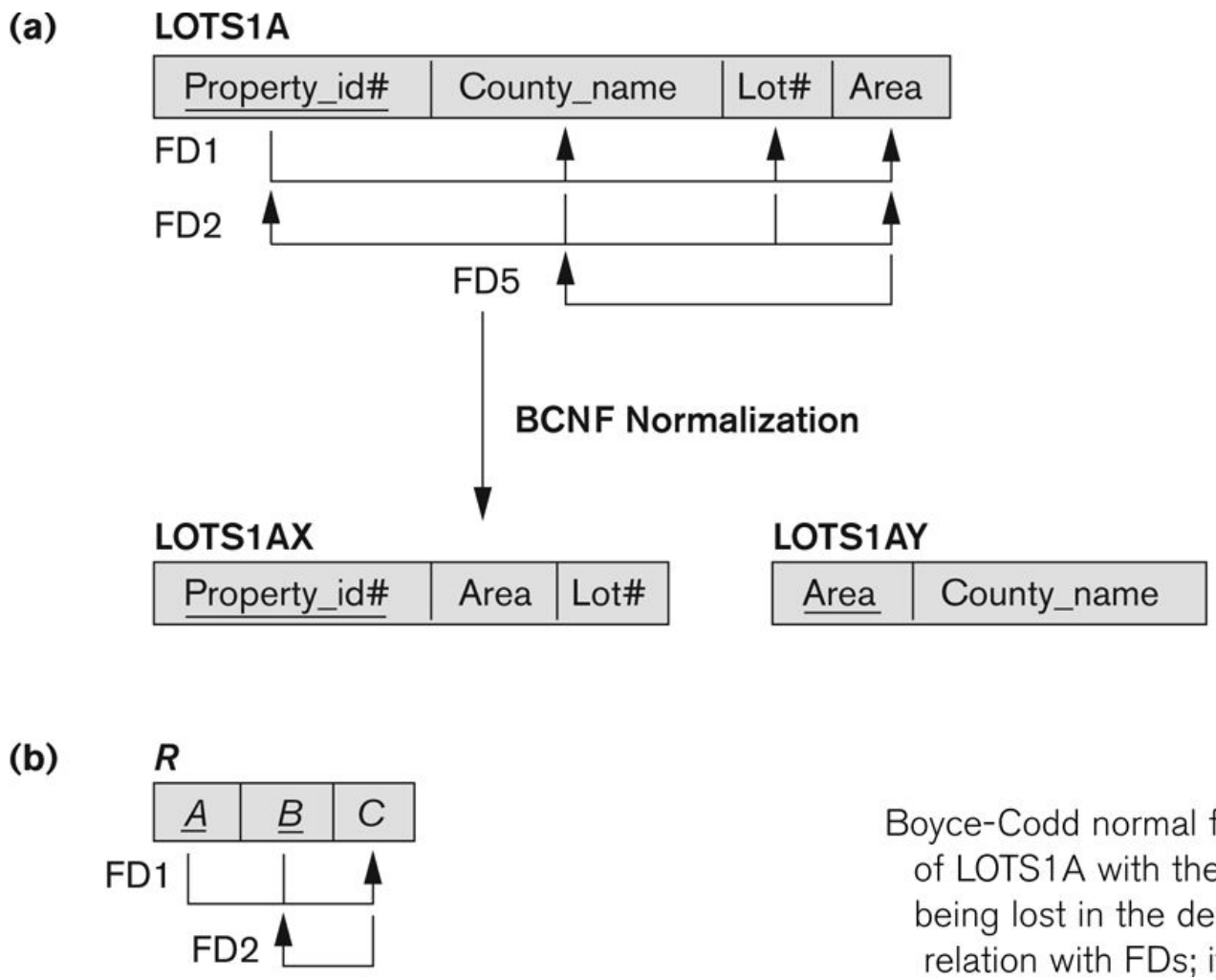


Figure 10.12
Boyce-Codd normal form. (a) BCNF normalization of LOTS1A with the functional dependency FD2 being lost in the decomposition. (b) A schematic relation with FDs; it is in 3NF, but not in BCNF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Consider the relation TEACH that is in 3NF but not in BCNF

- relation TEACH with the following dependencies:
  FD1: {Student,Course}→Instructor
  FD2: Instructor→Course

TEACH

Student	Course	Instructor
Narayan	Database	Mark
Smith	Database	Navathe
Smith	Operating Systems	Ammar
Smith	Theory	Schulman
Wallace	Database	Mark
Wallace	Operating Systems	Ahamad
Wong	Database	Omiecinski
Zelaya	Database	Navathe
Narayan	Operating Systems	Ammar

Figure 10.13
A relation TEACH that is in 3NF but not BCNF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Achieving the BCNF by Decomposition (1)

- Two FDs exist in the relation TEACH:
  - fd1: { student, course} -> instructor
  - fd2: instructor -> course
- {student, course} is a candidate key for this relation and that the dependencies shown follow the pattern in Figure 15.13 (b).
  - So this relation is in 3NF *but not in* BCNF
- A relation **NOT** in BCNF should be decomposed so as to meet this property, while possibly forgoing the preservation of all functional dependencies in the decomposed relations.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Achieving the BCNF by Decomposition (2)

- Three possible decompositions for relation TEACH
  - {student, instructor} and {student, course}
  - {course, instructor } and {course, student}
  - {instructor, course } and {instructor, student}
- All three decompositions will lose FD1.
  - We have to settle for sacrificing the functional dependency preservation. But we cannot sacrifice the non-additivity property after decomposition.
- Out of the above three, only the 3rd decomposition will not generate spurious tuples after join

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Consider the following relation for published books:

BOOK (Book_title, Author_name, Book_type, List_price, Author_affil, Publisher)

Author_affil refers to the affiliation of author. Suppose the following dependencies exist:

Book_title → Publisher, Book_type

Book_type → List_price

Author_name → Author_affil

a. What normal form is the relation in? Explain your answer.

b. Apply normalization until you cannot decompose the relations further. State the reasons behind each decomposition.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

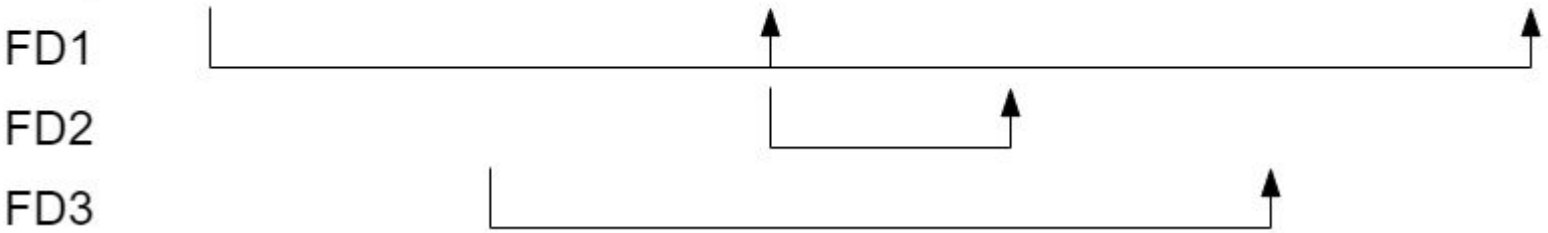
a. Since all attributes are directly or transitively dependent on either *Book_title* or *Author_name* attributes, we designate {***Book_title***, ***Author_name***} as the primary key.

Relation BOOK **is in first normal form** (1NF) because there are no relations inside relation and there are no multivalued attributes.

Relation BOOK **is not in second normal form** (2NF) because there are partial dependencies on the primary key. In fact, **each functional dependency given in this exercise is a partial dependency**.

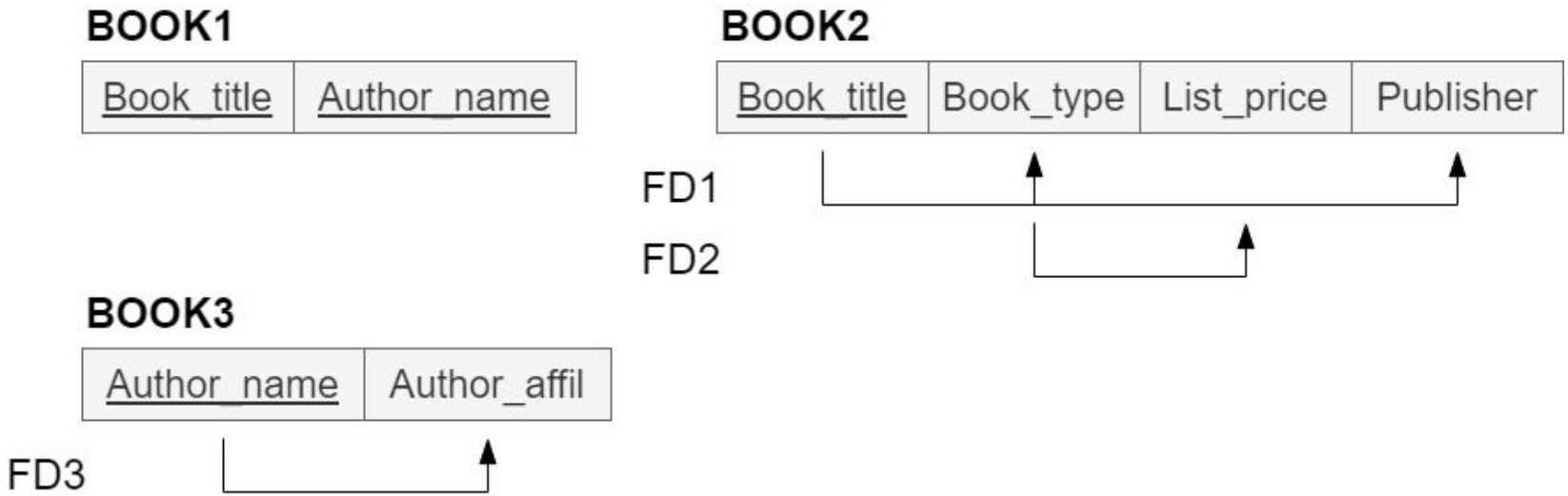
BOOK

Book_title	Author_name	Book_type	List_price	Author_affil	Publisher



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Decomposing it into 2NF



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Decomposing it into 3NF

BOOK1

Book_title	Author_name

BOOK2A

Book_title	Book_type	Publisher

FD1

BOOK2B

Book_type	List_price

FD2

BOOK3

Author_name	Author_affil

FD3

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

15.30. Consider the following relation:

CAR_SALE(Car#, Date_sold, Salesperson#, Commission%, Discount_amt)

Assume that a car may be sold by multiple salespeople, and hence {Car#, Salesperson#} is the primary key. Additional dependencies are

Date_sold → Discount_amt and

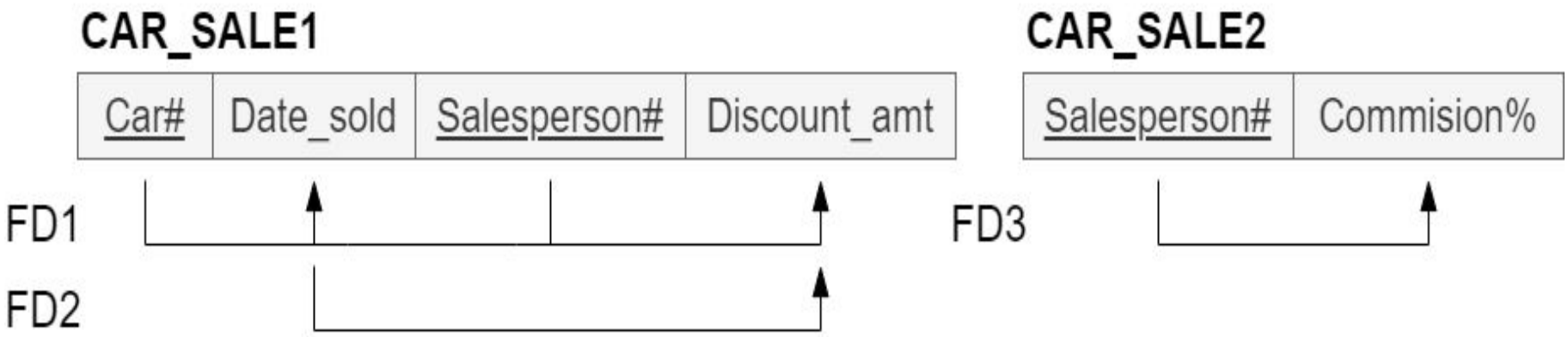
Salesperson# → Commission%

Based on the given primary key, is this relation in 1NF, 2NF, or 3NF? Why or why not? How would you successively normalize it completely?

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Firstly we notice that there is a single candidate key {Car#, Salesperson#} so we can use definitions of normal forms based on the primary key.
- In order for relation CAR SALE to be in second normal form we need to decompose it into two relations CAR_SALE1 and CAR_SALE2.
- This will eliminate partial dependencies on primary key.
- The primary key of relation CAR_SALE1 is {Car#, Salesperson#} and of CAR SALE2 is Salesperson#.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe



Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Properties of Relational Decompositions

Relation Decomposition and Insufficiency of Normal Forms

- The relational database design algorithms that we have seen so far has a **universal relation schema** R = {A1, A2, ..., An} that includes all the attributes of the database.
- We implicitly make the universal relation assumption, which states that every attribute name is **unique**.
- Using the functional dependencies, the algorithms decompose the universal relation schema R into a set of relation schemas D = {R1, R2, ..., Rm} that will become the relational database schema; D is called a **decomposition** of R.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- We must make sure that each attribute in R will appear in at least one relation schema Ri in the decomposition so that no attributes are lost; formally, we have

$$\bigcup_{i=1}^{m} R_i = R$$

- This is called the **attribute preservation condition** of a decomposition.
- Another goal is to have each individual relation Ri in the decomposition D be in BCNF or 3NF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- When relation is in BCNF and joined with a relation not in BCNF gives rise to *spurious tuples*
- To avoid this, we have additional conditions that should hold on a decomposition D are:
  1. Dependency Preservation Property of a Decomposition
  2. Nonadditive (Lossless) Join Property of a Decomposition
  3. Testing Binary Decompositions for the Nonadditive Join Property
  4. Successive Nonadditive Join Decompositions

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

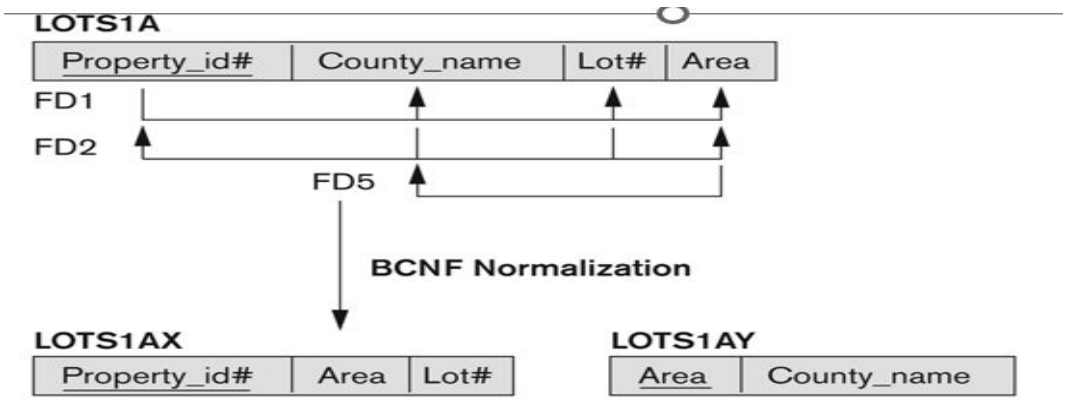
1. Dependency Preservation Property of a Decomposition

- It would be useful if each functional dependency X→Y specified in F either appeared directly in one of the relation schemas Ri in the decomposition D or could be inferred from the dependencies that appear in some Ri .
- Informally, this is the **dependency preservation condition**.

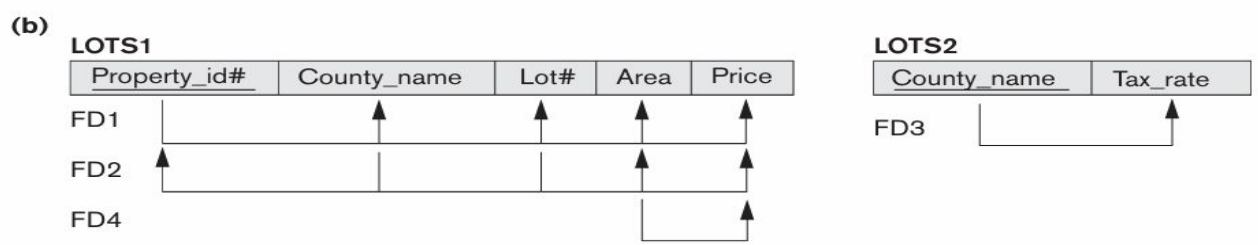
Definition. Given a set of dependencies F on R, the **projection** of F on R_i, denoted by $\pi_{Ri}(F)$ where R_i is a subset of R, is the set of dependencies $X \rightarrow Y$ in F^+ such that the attributes in $X \cup Y$ are all contained in R_i. Hence, the projection of F on each relation schema R_i in the decomposition D is the set of functional dependencies in F^+, the closure of F, such that all their left- and right-hand-side attributes are in R_i. We say that a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R is **dependency-preserving** with respect to F if the union of the projections of F on each R_i in D is equivalent to F; that is, $((\pi_{R_1}(F)) \cup \ldots \cup (\pi_{R_m}(F)))^+ = F^+$.

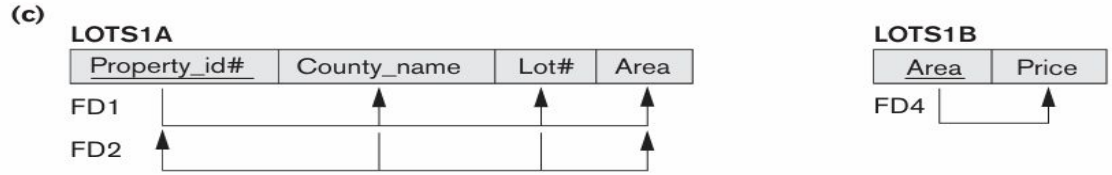
Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- For eg FD 2 is lost when LOTS is decomposed into {LOTS1AX, LOTS1AY} during the normalization process in previous slides



- But in the following it is preserved





Copyright © 2007 Ramez Elmasri and

- **Claim 1.** It is always possible to find a dependency-preserving decomposition D with respect to F such that each relation Ri in D is in 3NF.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

2. Nonadditive (Lossless) Join Property of a Decomposition

- Another property that a decomposition D should possess is the nonadditive join property, which ensures that **no spurious tuples are generated** when a NATURAL JOIN operation is applied to the relations resulting from the decomposition.

Definition. Formally, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the **lossless (nonadditive) join property** with respect to the set of dependencies F on R if, for *every* relation state r of R that satisfies F, the following holds, where $*$ is the NATURAL JOIN of all the relations in D: $*(\pi_{R_1}(r), \ldots, \pi_{R_m}(r)) = r$.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- If a decomposition does not have the lossless join property, we may get additional spurious tuples after the PROJECT (π) and NATURAL JOIN (*) operations are applied; these additional tuples represent erroneous or invalid information.
- Following is the algorithm for testing this property

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

Algorithm 16.3. Testing for Nonadditive Join Property

Input: A universal relation R, a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R, and a set F of functional dependencies.

Note: Explanatory comments are given at the end of some of the steps. They follow the format: (* *comment* *).

1. Create an initial matrix S with one row i for each relation R_i in D, and one column j for each attribute A_j in R.
2. Set $S(i, j) := b_{ij}$ for all matrix entries. (* each b_{ij} is a distinct symbol associated with indices (i, j) *).
3. For each row i representing relation schema R_i
   {for each column j representing attribute A_j
   {if (relation R_i includes attribute A_j) then set $S(i, j) := a_j$;};}; (* each a_j is a distinct symbol associated with index (j) *).

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

4. Repeat the following loop until a *complete loop execution* results in no changes to S

{for each functional dependency $X \rightarrow Y$ in F

{for all rows in S *that have the same symbols* in the columns corresponding to attributes in X

{make the symbols in each column that correspond to an attribute in Y be the same in all these rows as follows: If any of the rows has an *a* symbol for the column, set the other rows to that *same a* symbol in the column. If no *a* symbol exists for the attribute in any of the rows, choose one of the *b* symbols that appears in one of the rows for the attribute and set the other rows to that same *b* symbol in the column ;} ; } ;};

5. If a row is made up entirely of *a* symbols, then the decomposition has the nonadditive join property; otherwise, it does not.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Consider an example R = {A,B,C,D,E} is decomposed into R1(ABCD) and R2(DE)

If F= {A->B, A->C, A->D,D->E}. Find whether it is Lossy or Lossless

Step 1: Create the matrix S according to the algorithm

	A	B	C	D	E
R1	a1	a2	a3	a4	b15 a5
R2	b21	b22	b23	a4	a5

Since R1 row has all a's, the decomposition is lossless.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Consider R = {A,B,C,D} and decompositions R1(AD), R2(AB), R3(BE),R4(CDE),R5(AE). For the FDs

  F = {A->C, B->C, C->D,DE->C,CE->A} , using the algorithm find out the decomposition is lossy or lossless.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

3. Testing Binary Decompositions for the Nonadditive Join Property

- There is a special case of a decomposition called a binary decomposition—decomposition of a relation R into two relations.

Property NJB (Nonadditive Join Test for Binary Decompositions). A decomposition $D = \{R_1, R_2\}$ of R has the lossless (nonadditive) join property with respect to a set of functional dependencies F on R *if and only if* either

- The FD $((R_1 \cap R_2) \rightarrow (R_1 - R_2))$ is in F^+, or
- The FD $((R_1 \cap R_2) \rightarrow (R_2 - R_1))$ is in F^+

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

- Eg: R1(ABCD) and R2(DE) and F= {A->B, A->C, A->D,D->E}.
- R1 ∩ R2 -> R2-R1 that is D->E is in F+ hence the decomposition is lossless

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe

4. Successive Nonadditive Join Decompositions

- We saw the successive decomposition of relations during the process of second and third normalization in previous slides.
- To verify that these decompositions are nonadditive, we need to ensure another property, as set forth in Claim 2.

Claim 2 (Preservation of Nonadditivity in Successive Decompositions). If a decomposition $D = \{R_1, R_2, \ldots, R_m\}$ of R has the nonadditive (lossless) join property with respect to a set of functional dependencies F on R, and if a decomposition $D_i = \{Q_1, Q_2, \ldots, Q_k\}$ of R_i has the nonadditive join property with respect to the projection of F on R_i, then the decomposition $D_2 = \{R_1, R_2, \ldots, R_{i-1}, Q_1, Q_2, \ldots, Q_k, R_{i+1}, \ldots, R_m\}$ of R has the nonadditive join property with respect to F.

Copyright © 2007 Ramez Elmasri and Shamkant B. Navathe